



Carátula para entrega de prácticas

Facultad de Ingeniería

Laboratorios de docencia

Laboratorio de Computación Salas A y B

Profesor(a): Ariel Adara Mercado Martinez

Asignatura: Estructuras de datos y Algoritmos I

Grupo:

No de Práctica(s): postfija

Integrante(s): Chavez Sanchez Ehidan Javier

Salgado Sandoval Alexis Uriel

Villarruel Sánchez David Uriel

Castillo Vizcarra Ihsahn

No. de lista o 5
brigada:

Semestre: 2026-2

Fecha de entrega:

Observaciones:

CALIFICACIÓN: _____

Índice:

Índice:	2
Introducción:	2
Desarrollo	3
Conclusión:	10

Introducción:

Las expresiones matemáticas están metidas en todos lados en la informática. No importa si hablamos de una calculadora básica en el celular, de un programa científico más profesional o de un compilador sofisticado, las operaciones algebraicas son la base de todo. El detalle es que nosotros estamos acostumbrados a escribir y leer las otro formato, buscando que sea de la forma más natural posible para nosotros, metiendo los operadores en medio de los números y usando paréntesis para decidir qué va primero, como en el clásico $(a+b)*c$. Sin embargo, para una computadora esto no es tan fácil. Dejar las cosas así le exige al procesador estar regresándose a cada rato a revisar qué operador tiene más peso o dónde se abre y cierra un paréntesis, lo que vuelve el proceso lento y enredado.

Para solucionar este problema, en el mundo del software se usa muchísimo la notación postfija. En este formato, los operadores se colocan justo después de las variables, lo que permite que la máquina lea la expresión de corrido, de izquierda a derecha y en una sola pasada. Lo mejor de todo es que en la notación postfija los paréntesis desaparecen por completo y el orden de las operaciones queda perfectamente claro desde el principio.

El reto central de este proyecto consiste justamente en eso: crear un sistema capaz de leer un archivo de texto con variables, tomar esa expresión que está escrita de manera tradicional y traducirla correctamente a su equivalente postfijo, para después evaluarla y darnos el resultado numérico final. Para lograr que esto funcione de manera eficiente y sin límites fijos de espacio, la restricción del problema nos obliga a olvidarnos de los arreglos estáticos y construir estructuras dinámicas desde cero, utilizando específicamente pilas y colas ligadas.

Más allá de entregar un programa que funcione y pase las pruebas, el verdadero objetivo de la práctica es comprender a fondo cómo operan la precedencia y la asociatividad en un algoritmo real, aprender a manipular cadenas de texto y dominar el uso de archivos. A nivel de programación en C, este proyecto es un ejercicio excelente para soltarse con el manejo de la memoria RAM mediante apuntadores genéricos, entender los beneficios de separar el código en módulos independientes y ganar experiencia práctica en temas que son los pilares fundamentales de las estructuras de datos.

Desarrollo

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  #include "../include/cola.h"
6
7  Cola* crearCola() {
8      Cola *c = (Cola *)malloc(sizeof(Cola));
9      if (c == NULL) {
10         printf("Error: sin memoria para crear cola.\n");
11         return NULL;
12     }
13     c->frente = NULL;
14     c->final = NULL;
15     return c;
16 }
17
18 // Asigna memoria al puntero de la cola c
19
20 int colaVacia(Cola *c) {
21     return c->frente == NULL;
22 }
23
24 // Revisa si el frente de la pila apunta a NULL
25
26 void enqueue(Cola *c, void *dato, size_t size) {
27     NodoCola *nuevo = (NodoCola *)malloc(sizeof(NodoCola));
28     if (nuevo == NULL) {
29         printf("Error: sin memoria para enqueue.\n");
30         return;
31     }
32     nuevo->dato = malloc(size);
33     if (nuevo->dato == NULL) {
34         printf("Error: sin memoria para dato en enqueue.\n");
35         return;
36     }
37     memcpy(nuevo->dato, dato, size);
38     nuevo->siguiente = NULL;
39
40     if (colaVacia(c)) {
41         c->frente = nuevo;
42         c->final = nuevo;
43     } else {
44         c->final->siguiente = nuevo;
45         c->final = nuevo;
46     }
47 }
48
49 char dequeue(Cola *c) {
50     if (colaVacia(c)) {
51         printf("Error: dequeue en cola vacia.\n");
52         return '\0';
53     }
54     NodoCola *temp = c->frente;
55     c->frente = temp->siguiente;
56     if (c->frente == NULL)
57         c->final = NULL;
58
59     char dato = *((char *)temp->dato);
60     free(temp->dato);
61     free(temp);
62     return dato;
63 }
64
65 void imprimirCola(Cola *c) {
66     NodoCola *actual = c->frente;
67     while (actual != NULL) {
68         printf("%c", *((char *)actual->dato));
69         if (actual->siguiente != NULL)
70             printf(" ");
71         actual = actual->siguiente;
72     }
73     printf("\n");
74 }
75
76 void destruirCola(Cola *c) {
77     while (!colaVacia(c)) {
78         NodoCola *temp = c->frente;
79         c->frente = temp->siguiente;
80         free(temp->dato);
81         free(temp);
82     }
83     free(c);
84 }
```

La cola es una estructura clave del proyecto y está construida con una lista enlazada y memoria dinámica. La cola funciona como una fila donde los elementos entran por un extremo y salen por el otro, respetando el orden en que llegaron. Eso la hace ideal para guardar la expresión postfija, porque los tokens necesitan conservar exactamente el orden en que fueron generados.

Para hacer eso posible, la estructura lleva dos punteros: uno al frente, que es por donde salen los elementos, y otro al final, que es por donde entran. Cuando se agrega un elemento con enqueue, se crea un nuevo nodo, se le copia el dato y se engancha al final de la fila. Si la cola estaba vacía en ese momento, ese nodo se convierte simultáneamente en el frente y en el final. Cuando se extrae un elemento con dequeue, se toma desde el frente, se libera su memoria y se avanza el puntero al siguiente. Si la cola queda vacía tras esa extracción, el puntero al final también se limpia para evitar que quede apuntando a memoria liberada.

También hay una función imprimirCola que recorre toda la cola de principio a fin e imprime cada token separado por un espacio, útil para ver en pantalla cómo quedó la expresión postfixa. Y al final del programa, destruirCola se encarga de liberar todo lo que quedó en memoria antes de cerrar.

Dentro del proyecto, la cola actúa como el puente entre la conversión y la evaluación: recibe los tokens en el orden correcto y los entrega en ese mismo orden cuando llega el momento de calcular el resultado.

main.c

```
Code Blame 45 lines (31 loc) · 775 Bytes

1  #include <stdio.h>
2
3  #include "../include/parser.h"
4  #include "../include/evaluator.h"
5  #include "../include/cola.h"
6
7  int main() {
8
9      Variable variables[MAX_VARIABLES];
10
11     int totalVariables;
12
13     char expresion[MAX_EXPR];
14
15     cargarArchivo(
16         "input/casol.txt",
17         variables,
18         &totalVariables,
19         expresion
20     );
21
22     printf("Expresion infija:\n");
23     printf("%s\n\n", expresion);
24
25     Cola* postfija =
26         infijaAPostfija(expresion);
27
28     printf("Expresion postfija:\n");
29
30     imprimirCola(postfija);
31
32     float resultado =
33         evaluarPostfija(
34             postfija,
35             variables,
36             totalVariables
37         );
38
39     printf("\nResultado:\n");
40     printf("%.2f\n", resultado);
41
42     destruirCola(postfija);
43 }
```

Aquí main.c es el punto de entrada del programa, pero no hace casi nada por su cuenta. Su verdadero papel es conectar todos los módulos en el orden correcto para que el proceso fluya de principio a fin. Todo comienza leyendo el archivo de entrada con cargarArchivo, que extrae dos cosas: los valores de las variables definidas y la expresión matemática en notación infija. Con eso en mano, el programa ya tiene todo lo que necesita para trabajar.

El siguiente paso es la conversión. La expresión infija se le pasa a `infijaAPostfija`, que la transforma en una cola de tokens ordenados en notación postfija. Ambas versiones de la expresión se imprimen en pantalla, lo que permite ver claramente cómo cambió la estructura después de la conversión.

Con la expresión ya en formato postfija, se llama a `evaluarPostfija`, que la recorre, aplica las operaciones usando una pila y devuelve el resultado como un número decimal. Ese resultado es el que finalmente se muestra en pantalla.

Antes de cerrar, el programa libera la memoria que usó la cola, algo importante para no dejar recursos ocupados innecesariamente.

`main.c` no resuelve nada por sí solo, simplemente se asegura de que cada parte del programa entre en acción en el momento indicado, como si fuera el director de una obra donde los demás módulos son los que realmente actúan.

parser.c

```

1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  #include "../include/parser.h"
6  #include "../include/pila.h"
7  #include "../include/cola.h"
8
9  void cargarArchivo(
10     const char* nombreArchivo,
11     Variable variables[],
12     int* totalVariables,
13     char expresion[]
14 ) {
15     FILE *arch = fopen(nombreArchivo, "r");
16     if (arch == NULL) {
17         printf("yo creo que no se va a poder %s.\n", nombreArchivo);
18         return;
19     }
20
21     char linea[MAX_EXPR];
22     *totalVariables = 0;
23
24     while (fgets(linea, sizeof(linea), arch) != NULL) {
25         int longitud = strlen(linea);
26         if (longitud > 0 && linea[longitud - 1] == '\n') {
27             linea[longitud - 1] = '\0';
28         }
29
30         if (strlen(linea) == 0) continue;
31
32         int igual = 0;
33         for (int i = 0; i < strlen(linea); i++) {
34             if (linea[i] == '=') {
35                 igual = 1;
36                 break;
37             }
38         }
39
40         if (igual == 1) {
41             char *nombre = strtok(linea, " =");
42             char *numtext = strtok(NULL, " =");
43

```

```

44         variables[*totalVariables].nombre = nombre[0];
45         variables[*totalVariables].valor = atof(numtext);
46         *totalVariables = *totalVariables + 1;
47     } else {
48         strcpy(expresion, linea);
49     }
50 }
51
52 fclose(arch);
53 }
54
55 int esOperador(char c) {
56     if (c == '+' || c == '-' || c == '*' || c == '/' || c == '^') {
57         return 1;
58     }
59     return 0;
60 }
61
62 int precedencia(char op) {
63     if (op == '^') {
64         return 3;
65     } else if (op == '*' || op == '/') {
66         return 2;
67     } else if (op == '+' || op == '-') {
68         return 1;
69     }
70     return 0;
71 }
72
73 Cola* infijaAPostfija(char* expresion) {
74     Cola *postfija = crearCola();
75     Pila *pila = crearPila();
76
77     int longitud = strlen(expresion);
78
79     for (int i = 0; i < longitud; i++) {
80         char actual = expresion[i];
81
82         if (actual == ' ') {
83             continue;

```

```

82     if (actual == ' ') {
83         continue;
84     }
85
86     if ((actual >= 'a' && actual <= 'z') || (actual >= 'A' && actual <= 'Z')) {
87         enqueue(postfija, &actual, sizeof(char));
88     }
89     else if (actual == '(') {
90         push(pila, &actual, sizeof(char));
91     }
92     else if (actual == ')') {
93         while (pilaVacia(pila) == 0 && peek(pila) != '(') {
94             char op = pop(pila);
95             enqueue(postfija, &op, sizeof(char));
96         }
97         if (pilaVacia(pila) == 0) {
98             pop(pila);
99         }
100     }
101     else if (esOperador(actual) == 1) {
102         while (pilaVacia(pila) == 0 && peek(pila) != '(') {
103             char tope = peek(pila);
104             int ant = precedencia(actual);
105             int precTope = precedencia(tope);
106
107             int sacar = 0;
108
109             if (actual != '^' && precTope >= ant) {
110                 sacar = 1;
111             } else if (actual == '^' && precTope > ant) {
112                 sacar = 1;
113             }
114
115             if (sacar == 1) {
116                 char op = pop(pila);
117                 enqueue(postfija, &op, sizeof(char));
118             } else {
119                 break;
120             }
121         }
122         push(pila, &actual, sizeof(char));
123     }
124 }
125
126 while (pilaVacia(pila) == 0) {
127     char op = pop(pila);
128     enqueue(postfija, &op, sizeof(char));
129 }
130
131 destruirPila(pila);
132 return postfija;
133 }

```

El archivo parser.c tiene dos trabajos principales: leer los datos desde un archivo de texto y convertir la expresión matemática de notación infija a postfija. La lectura la hace cargarArchivo, que recorre el archivo línea por línea distinguiendo dos tipos de contenido: si una línea tiene un signo = es una variable con su valor, y si no, es la expresión matemática. Con un solo recorrido ya tiene todo listo para continuar.

Para hacer la conversión correctamente, el programa primero necesita entender la jerarquía de los operadores. La función precedencia le asigna un nivel numérico a cada uno: la potencia tiene prioridad más alta, luego la multiplicación y división, y al final la suma y resta. Junto con eso, la asociatividad define qué pasa cuando dos operadores tienen el mismo nivel: casi todos se resuelven de izquierda a derecha, excepto la potencia, que se resuelve de derecha a izquierda. Esa diferencia tiene su propio tratamiento especial en el código.

Con esas reglas claras, infijaAPostfija recorre la expresión carácter por carácter tomando decisiones según lo que encuentra. Las letras pasan directo a la cola de salida, los paréntesis que abren se apilan y esperan, y cuando aparece un paréntesis que cierra se vacían los operadores acumulados hasta encontrar su par de apertura. Cuando aparece un operador, se compara con el del tope de la pila y si ese tiene mayor o igual precedencia sale primero. Al terminar, lo que quede en la pila se vuelca a la cola. El resultado es la expresión en notación postfija lista para evaluarse, por ejemplo $(a + b) * c$ se convierte en $a b + c *$.

pila.c

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <string.h>
4
5  #include "../include/pila.h"
6
7  Pila* crearPila() {
8      Pila *p = (Pila *)malloc(sizeof(Pila));
9      if (p == NULL) {
10         printf("Error: sin memoria para crear pila.\n");
11         return NULL;
12     }
13     p->tope = NULL;
14     return p;
15 }
16
17 // Se reserva memoria para el TDA "pila" y se verifica que no este vacia, si no lo esta, se asigna NULL al tope
18
19 int pilaVacia(Pila *p) {
20     return p->tope == NULL;
21 }
22
23 void push(Pila *p, void *dato, size_t size) {
24     NodoPila *nuevo = (NodoPila *)malloc(sizeof(NodoPila));
25     if (nuevo == NULL) {
26         printf("Error: sin memoria para push.\n");
27         return;
28     }
29     nuevo->dato = malloc(size);
30     if (nuevo->dato == NULL) {
31         printf("Error: sin memoria para dato en push.\n");
32         return;
33     }
34     memcpy(nuevo->dato, dato, size);
35     nuevo->siguiente = p->tope;
36     p->tope = nuevo;
37 }
38
39 // Ingresa un nuevo elemento en la cima de la pila
40
41 char pop(Pila *p) {
42     if (pilaVacia(p)) {
43         printf("Error: pop en pila vacia.\n");
44         return '\0';
45     }
46 }
```

```

44     return '\0';
45 }
46 NodoPila *temp = p->tope;
47 p->tope = temp->siguiente;
48 char dato = *((char *)temp->dato);
49 free(temp->dato);
50 free(temp);
51 return dato;
52 }
53
54 char peek(Pila *p) {
55     if (pilaVacia(p)) return '\0';
56     return *((char *)p->tope->dato);
57 }
58
59 void destruirPila(Pila *p) {
60     while (!pilaVacia(p)) {
61         NodoPila *temp = p->tope;
62         p->tope = temp->siguiente;
63         free(temp->dato);
64         free(temp);
65     }
66     free(p);
67 }
68

```

La pila es una de las estructuras principales del proyecto y está implementada en `pila.c` usando una lista enlazada simple. En lugar de reservar un bloque fijo de memoria desde el inicio, cada elemento se crea y destruye dinámicamente conforme se necesita, lo que le da a la estructura la flexibilidad de crecer o reducirse según el flujo del programa.

Cada nodo de la lista guarda un puntero al dato almacenado y otro al nodo que está debajo de él en la pila. Cuando se inserta un elemento con `push`, se reserva memoria tanto para el nodo como para el dato en sí, y se copia su contenido usando `memcpy`. Gracias a que el dato se maneja como `void *`, la función puede recibir cualquier tipo de valor, no solo caracteres. Cuando se extrae un elemento con `pop`, se libera esa memoria y se retorna el valor. Además, existe `peek` para consultar el tope sin modificar la pila, algo que resulta muy útil al momento de decidir qué hacer con un operador sin todavía descartar. Al terminar, `destruirPila` se encarga de recorrer todos los nodos que quedaron y liberar su memoria uno por uno, evitando fugas que podrían acumularse durante la ejecución.

Conclusión individual

Este fue, sin duda, un proyecto extenso que no solo fomentó la cooperación y el trabajo en equipo, sino que también nos enfrentó al reto de gestionar un software de mayor escala. Aunque la complejidad del programa inicialmente fue alta, la estrategia de segmentar el problema por partes y profundizar tanto en los conceptos vistos en clase como en investigación complementaria, nos permitió digerir el desarrollo de manera estructurada. Al final, este proceso no solo dio como resultado un sistema funcional, sino que nos brindó una comprensión mucho más sólida y profunda de la materia.

Conclusión del equipo

Al arrancar el proyecto, el algoritmo parecía bastante claro. Pero en el momento de hacerlo en C aparecieron detalles que no se ven a primera vista: cuándo exactamente sacar un operador de la pila, cómo manejar los paréntesis sin romper el orden, por qué la potencia necesita un tratamiento distinto al resto. Cada una de esas pequeñas decisiones requirió detenerse a pensar con más cuidado de lo esperado. Usar listas enlazadas con memoria dinámica en lugar de arreglos fijos fue una decisión que complicó las cosas al inicio, pero que con el tiempo demostró tener sentido. Manejar `malloc` y `free` manualmente

enseña a ser muy consciente de lo que pasa en memoria en cada momento, algo que no se aprende del mismo modo leyendo un libro. Más de una vez un error difícil de rastrear resultó ser simplemente un puntero mal liberado o un nodo olvidado.

La modularización también fue un desafío al principio, porque separar el código en archivos independientes obliga a pensar en qué le corresponde hacer a cada parte antes de empezar a escribirla. Pero una vez que cada módulo tuvo su lugar claro, depurar y hacer ajustes se volvió mucho más sencillo. Lo que quedó al final fue algo más que un programa funcionando. Quedó una comprensión real de por qué estas estructuras existen y para qué sirven. Ver que una pila y una cola, bien usadas, son suficientes para resolver un problema tan concreto como convertir y evaluar expresiones matemáticas fue, honestamente, bastante satisfactorio.